

武汉大学弘毅学堂

本科生实验报告

高级语言程序设计综合实验 1

专 业 名 称 : 工科试验班

课 程 名 称 : 高级语言程序设计实验

指 导 教 师 : 谭成予 副教授

学 生 学 号 : 2022302191502

学 生 姓 名 : 刘瑾修

学 生 学 号 : 2022302191708

学 生 姓 名 : 高建宇

二〇二二年十一月

郑 重 声 明

本人呈交的实验报告，是在指导老师的指导下，独立进行实验工作所取得的成果，所有数据、图片资料真实可靠。尽我所知，除文中已经注明引用的内容外，本实验报告不包含他人享有著作权的内容。对本实验报告做出贡献的其他个人和集体，均已在文中以明确的方式标明。本实验报告的知识产权归属于培养单位。

本人签名： 刘瑾修

日期： 2022.11.20

本人签名： 高建宇

日期： 2023.2.08

摘 要

高级语言程序设计实验的实验目的是锻炼高级语言的使用能力。

实验设计主要遵循题目要求、面向对象的思想及测试驱动开发（Testdriven Development， TDD）的软件开发方法

实验内容主要包括：类拳皇的格斗游戏

实验结论为实验成功。

关键词：C/C++；高级语言程序设计；横屏格斗游戏

目录

摘 要	3
1 实验目的和意义	1
1.1 实验目的	1
1.2 实验意义	1
2 实验设计	2
2.1 概述	2
2.2.1 概要设计	2
2.2.2 数据结构	2
2.2.2 UML 类图	3
2.2.3 实例分析	4
3 运行	9
3.1 实验结果	9
3.2 实验介绍	9
4 参考文献	13

1 实验目的和意义

1.1 实验目的

本实验的目的是通过编码和测试指定问题,综合运用所学高级语言程序设计和游戏引擎的知识,提高熟练使用高级语言编写代码和解决问题的能力,锻炼使用高级语言程序设计的思维。

1.2 实验意义

希望通过独立开发一个游戏,完成开发从无到有的全过程,增加自己对于高级语言的熟悉程度,并加深自己对于 C++ 的理解,和程序开发的理解。为后面进行学习打下坚实的基础。

2 实验设计

2.1 概述

拳皇是一款经典的横屏街机游戏，受到很多人的喜爱，拳皇这个 IP 虽然被不断翻新，但是已经失去了原本的味道，我们希望通过这个实验，既能够锻炼到自己的程序开发能力，也能复刻出自己记忆中的拳皇。

2.2.1 概要设计

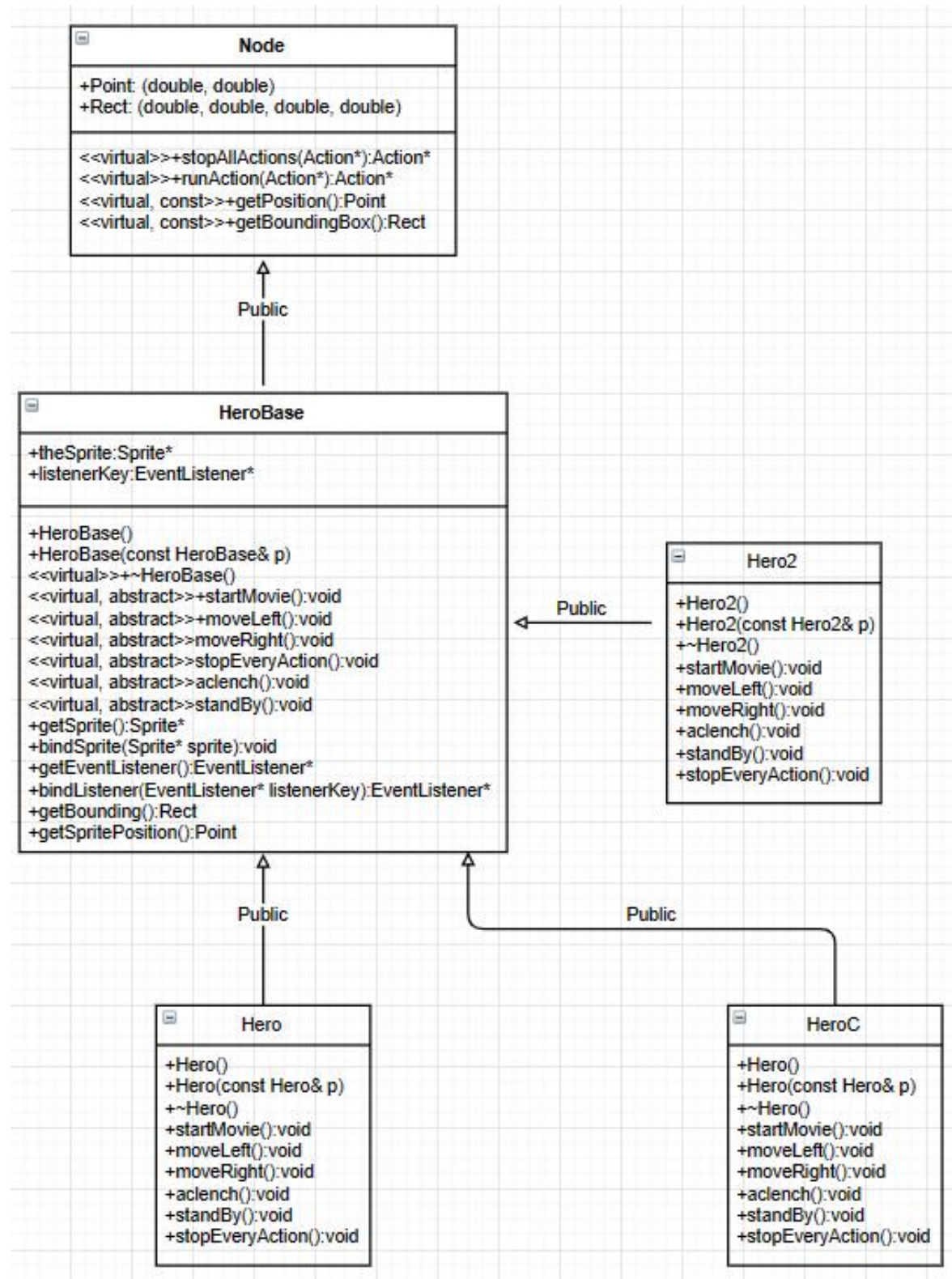
在经过开始界面的设置之后，玩家会进入到用户选择界面，选择自己喜欢的角色，在选定完角色之后会进入到战斗界面，直到玩家中有一方血条为零时战斗结束并判定胜负。

2.2.2 数据结构

本程序使用一些变量来储存程序中的数据。

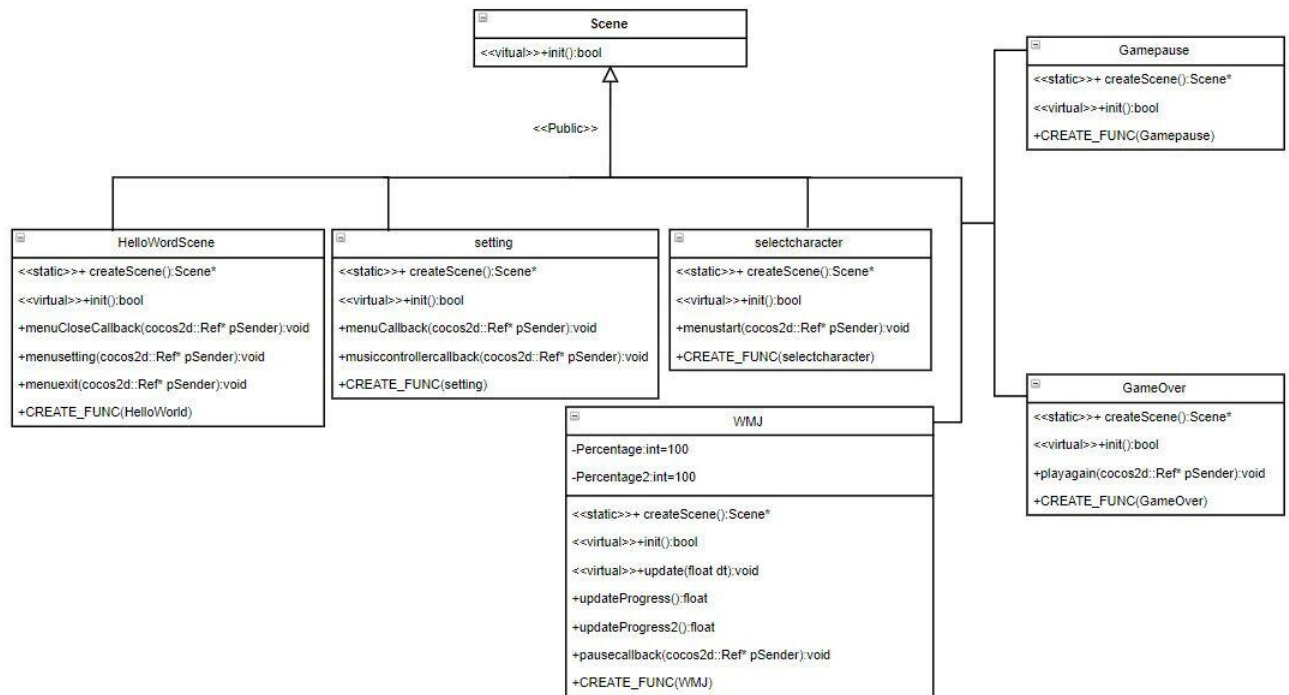
2.2.2 UML 类图

//刘瑾修负责部分



HeroBase 类是一个虚基类，其三个子类分别为三个人物角色，对所有角色都相同的操作我将在基类中进行定义，而每个子类的攻击动作等都是不同的，需要调用不同的帧动画，因此使用虚函数来在每个子类中重写该子类对应角色的函数。

//高建宇负责部分



场景(Scene) 是一个容器，容纳游戏中的各个元素，如精灵，标签，节点对象。它负责着游戏的运行逻辑，以帧为单位渲染内容。

2.2.3 实例分析

//刘瑾修负责部分

碰撞检测

```

void Hero::acLench()
{
    theSprite->stopAllActions();
    auto callBackFunc1 = [&]() {
        listenerKey->setEnabled(false);
    };
    CallFunc* callBack1 = CallFunc::create(callBackFunc1);
    auto callBackFunc2 = [&]() {
        listenerKey->setEnabled(true);
    };
    CallFunc* callBack2 = CallFunc::create(callBackFunc2);
}
  
```

```

Vector<SpriteFrame*> animFramesClench;
animFramesClench.reserve(5);
for (int i = 1; i <= 5; i++) { animFramesClench.pushBack(SpriteFrame::create
    ("clench" + to_string(i) + ".png", Rect(-12, 5, 160, 137))); }
Animation* animation = Animation::createWithSpriteFrames(animFramesClench, 0.1f);
Animate* animateBack1 = Animate::create(animation);
theSprite->stopAllActions();
auto seq11 = Sequence::create(callBack1, Repeat::create
    (animateBack1, 1), callBack2, nullptr);
theSprite->runAction(seq11);
}

```

玩家在按下 J 键时角色进行的攻击动作实际上是一个有 5 帧的帧动画,但是在播放攻击的帧动画时会先对 theSprite 使用 stopAllActions() 函数,让 theSprite 停止移动与正在播放的帧动画(一般会站立时的帧动画)之后再播放攻击的帧动画,因为同时播放两个帧动画会使画面紊乱,而玩家在松开 J 键时,theSprite 又会调用一次 stopAllAction(),如果玩家松开得过快的话会使得攻击动作强行停止并播放原本的站立帧动画,且在松开后玩家可以迅速按下第二次,并进行第二次伤害判定和帧动画播放。也就是说在这种情况下,理论上只要玩家的手速足够快,可以中断本次攻击直接进行下一次攻击和攻击判定,这是一个 bug。

为了解决这个问题,我们在 theSprite 要执行的 sequence 动作中加入了 call 函数。sequence 中全部是指针变量,指向要执行的动作,在 sequence 中的指针是按顺序调用的,只用在在前一个指针指向的代码行执行完成后才会使 theSprite 执行下一个指针中包含的动作。我们在攻击的帧动画前面加入了一个 call 函数使得 theSprite 失去对键盘的监听,这时如果玩家松开 J 键,动作也会执行完毕,不会停止,直至动画播放完毕,才会执行第二个 call 函数恢复对键盘事件的监听。

键盘监听

```

if ((keyCode == EventKeyboard::KeyCode::KEY_1
    || keyCode == EventKeyboard::KeyCode::KEY_2) && bashen2 == 1) {...}

```

这是对八神庵角色在松开键盘时的事件监听,加入 “&& bashen2 == 1” 的原因是区别对 P1 玩家和 P2 玩家的键盘事件的监听,如果不加以区分,每个两个人物都监听 A、D、1、2 四个键的话,当一名玩家松开键盘时,另一名玩家也会监听到这一事件,导致动作不流畅,无法实现想要的效果。

双人游戏

```

void selectcharacter::menustart(Ref* pSender) {
    bool P1 = bashen1 || huowu1 || dashe1;
    bool P2 = bashen2 || huowu2 || dashe2;
    if(P1&&P2)
        Director::getInstance()->replaceScene(WMJ::create());
    else {
        auto label2 = Label::createWithSystemFont("you need enough characters to start the

```

```
game!", "Consolas", 25);
    label2->setPosition(Vec2(600, 150));
    this->addChild(label2);
}
}
```

这是用于从角色选择界面切换到战斗界面的按钮的实现,在此处我利用选择角色时传递的参数构建了两个 bool 值来代表两名玩家角色选择的状态,如果有玩家没有选择角色,那么经过 if 判断则无法进入战斗场景,并会输出一段文字提醒玩家需要选择足够的角色开始游戏。

//高建宇负责部分

声音开关

首先先设置一个全局变量 `soundflag`, 以便于各个场景之间声音的开关状态能够共通

```
extern bool soundflag;
```

在设置场景中, 为营造出音量随按钮开关实时变化, 用了 `AudioEngine::pauseAll()` 和 `AudioEngine::resumeAll()` 函数来进行声音控制, 如果用 `setEnabled()` 函数则会导致每次切换时背景音乐都从头开始播放。

```
void setting::musiccontrollercallback(cocos2d::Ref* pSender)
{
    bool bSound = soundflag;
    bSound = !bSound;
    soundflag = bSound;
    if (bSound) {
        AudioEngine::resumeAll();
    }
    else {
        AudioEngine::pauseAll();
    }
}
```

在非设置场景中, 声音的控制则可以用 `setEnabled()`, 而不会让人感到不自然

```
AudioEngine::setEnabled(soundflag);
```

角色选择与创建

角色选择创建分为两个场景来进行实现, 在第一个场景中根据按下的按钮来给之前写好的全局变量赋值, 如下:

```
buttonBaShen1->addTouchEventEventListener([&](Ref* sender, Widget::TouchEvent type) {
    bashen1 = 1;
    dashe1 = 0;
    huowu1 = 0;
});
```

在战斗场景中我们再根据角色选择场景设定的全局变量的值来进行对应角色的创建

```

if (bashen1 == 1) {
    this->addChild(test1);
    test1->startMovie();
}
else if (bashen2 == 1) {
    test1->setPosition(Vec2(visibleSize.width / 2 + origin.x + 300, visibleSize.height
/ 2 + origin.y - 180));
    test1->setScaleX(-2);
    this->addChild(test1);
    test1->startMovie();
}

```

利用设置全局变量来传递参数进行角色选择与对应角色创建有许多优点，首先是简单易懂，其次不容易造成角色重复生成或因为按钮多按或者按不同的按钮而导致的一次生成多个角色。如果用按钮直接进行角色创建，会出现许多麻烦，特别是在重新开始游戏时容易出现上一个创建的角色不会被清除的现象。在添加了下述的代码后，角色的重复生成就会被解决。

```

else if (bashen1 == 0 || bashen2 == 0)
{
    this->removeChild(test1);
}

```

胜负判定

胜负判定实现的方法也有用到全局变量传递参数，来创建 P1 胜利与 P2 胜利的精灵

```

auto Plsprite = Sprite::create("UI/1P.png");
Plsprite->setPosition(winSize / 2);
if (Plwin = true) {
    this->addChild(Plsprite);
}
else{
    this->removeChild(Plsprite);
}

auto P2sprite = Sprite::create("UI/2P.png");
P2sprite->setPosition(winSize / 2);

if (P2win=true) {
    this->addChild(P2sprite);
}
else {
    this->removeChild(P2sprite);
}

```

胜负判定还用到了实时判断角色血量来设置参数、切换场景的函数 update()

```

void WMJ::update(float dt)
{
    if (Percentage2 <= 0)
    {

```

```

        Scene* scene = Scene::create();
        scene->addChild(GameOver::create());
        Director::getInstance()->replaceScene(GameOver::create());
        P2win = true;
        P1win = false;
    }
    else {}
    if (Percentage <= 0)
    {
        Scene* scene = Scene::create();
        scene->addChild(GameOver::create());
        Director::getInstance()->replaceScene(GameOver::create());
        P1win = true;
        P2win = false;
    }
    else {}
}

```

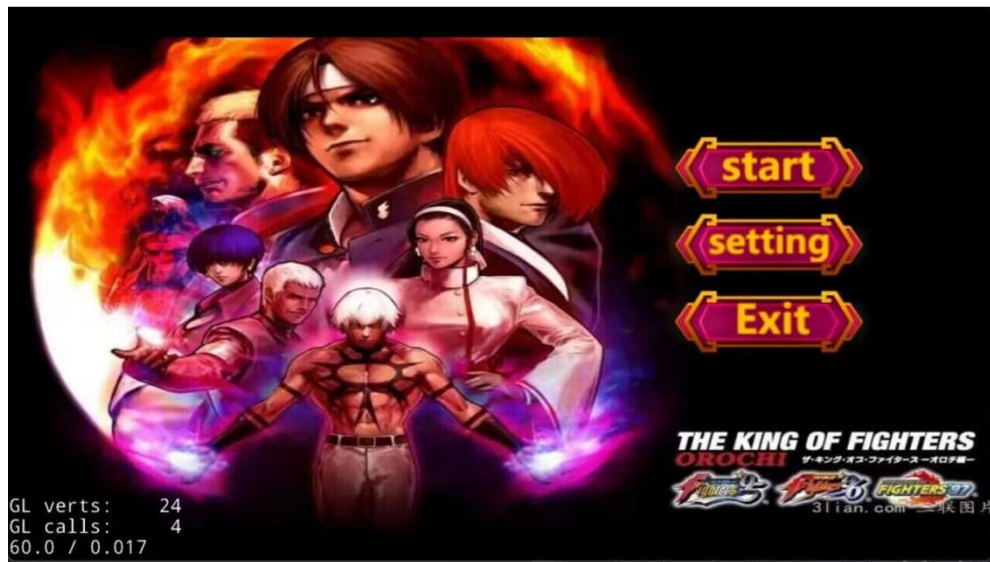
3 运行

3.1 实验结果

程序正常运行。

3.2 实验介绍

3.2.1 主页



点击 **start** 按钮会跳转至角色选择页面，点击 **setting** 按钮会跳转至音量设置界面，点击 **Exit** 按钮会退出程序。

3.2.2 音量设置



点击后移动滑槽来调整音量大小，调整完毕后点击 Back 按钮返回主页。

3.2.3 角色选择



点击角色头像选择角色，两位玩家不可选择相同角色。

两名玩家选择角色完毕之后点击 start 按钮即可进入战斗场景，开始游戏。

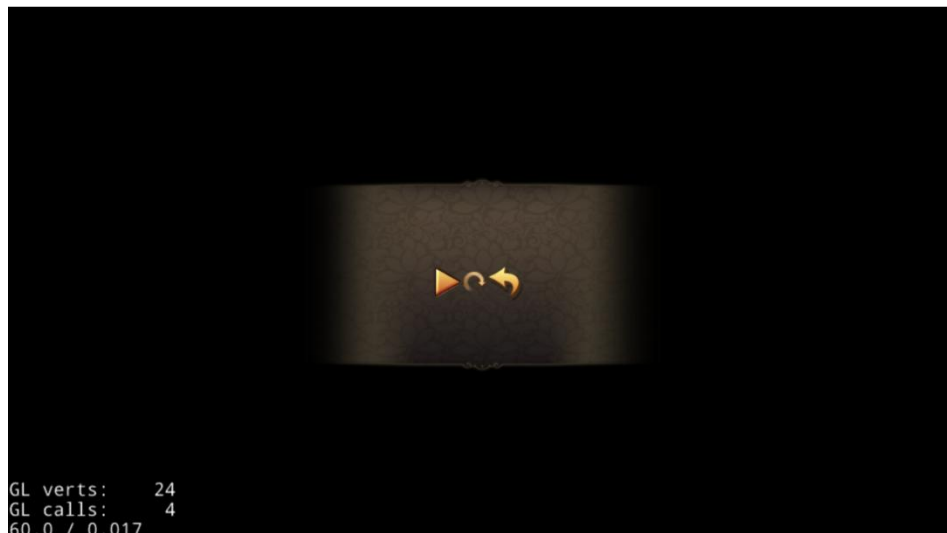
3.2.4 战斗场景



位于屏幕左边的是玩家 P1，位于屏幕右边的是玩家 P2

分别位于屏幕上方两端的两个红色长方形是玩家 P1 与玩家 P2 的血条，当玩家进行攻击时会进行检测，两个角色之间是否存在碰撞，如果检测到碰撞则会减少被攻击方的血条长度，当血条扣完时游戏角色死亡，游戏结束。

3.2.5 暂停场景



暂停场景主要由三个按钮组成：继续，返回选择角色场景和返回主场景。

3.2.6 结束场景



结束场景含有两个切换场景的按钮，分别切换至角色选择场景和主场景，并根据战斗场景中的参数来判断胜负

4 参考文献

钟迪龙：《Cocos2d-x 3.x 游戏开发之旅》 电子工业出版社

Cocos2dx 官方网站教程 <https://docs.cocos.com/cocos2d-x/manual/zh/>

B 站中的教程

https://www.bilibili.com/video/BV1UT4y1A7GN/?spm_id_from=333.337.search-card.all.click

https://www.bilibili.com/video/BV1g34y1n7HQ/?spm_id_from=333.337.search-card.all.click